Purpose of Creating a Series in Different Ways

In Python, data can exist in different formats such as lists, tuples, dictionaries, NumPy arrays, or ranges.

The purpose of these methods is to convert different types of data into a Pandas Series so that it can be easily analyzed, manipulated, and processed using Pandas functions.

A Series is basically a one-dimensional labeled array that stores data along with an index.

"We use different methods to create a Series because real-world data comes in different formats, and Pandas allows us to convert all those formats into a structured form for data analysis."

A Pandas Series is a one-dimensional labeled array capable of holding any type of data such as integers, strings, or floats. Each value in a Series has an associated index.

Brief Intro of Each Method

1. Series Using List

A list stores ordered data.

When we create a Series from a list, each element of the list becomes a value in the Series.

Purpose:

Used when data is available in a simple sequential collection.

2. Series Using List with Custom Index

By default, Series uses numeric indexes (0,1,2...).

But we can assign custom labels to identify data more clearly.

Purpose:

Used when we want meaningful labels instead of numeric indexes (like student names, subjects, etc.).

3. Series Using Tuple

A tuple is similar to a list but immutable (cannot be changed).

Purpose:

Used when the data should remain constant and not be modified.

4. Series Using Dictionary

In a dictionary, data is stored as key–value pairs.

When creating a Series from a dictionary, keys become indexes and values become data.

Purpose:

Useful when data already has labels associated with it.

5. Series Using NumPy Array

A NumPy array is designed for fast numerical computations.

Purpose:

Used in data science and machine learning when working with large numerical datasets.

6. Series Using Range

The `range()` function generates a sequence of numbers.

Purpose:

Used when we want to quickly generate a series of numbers without manually writing them.

Simple explanation :::::::::>>

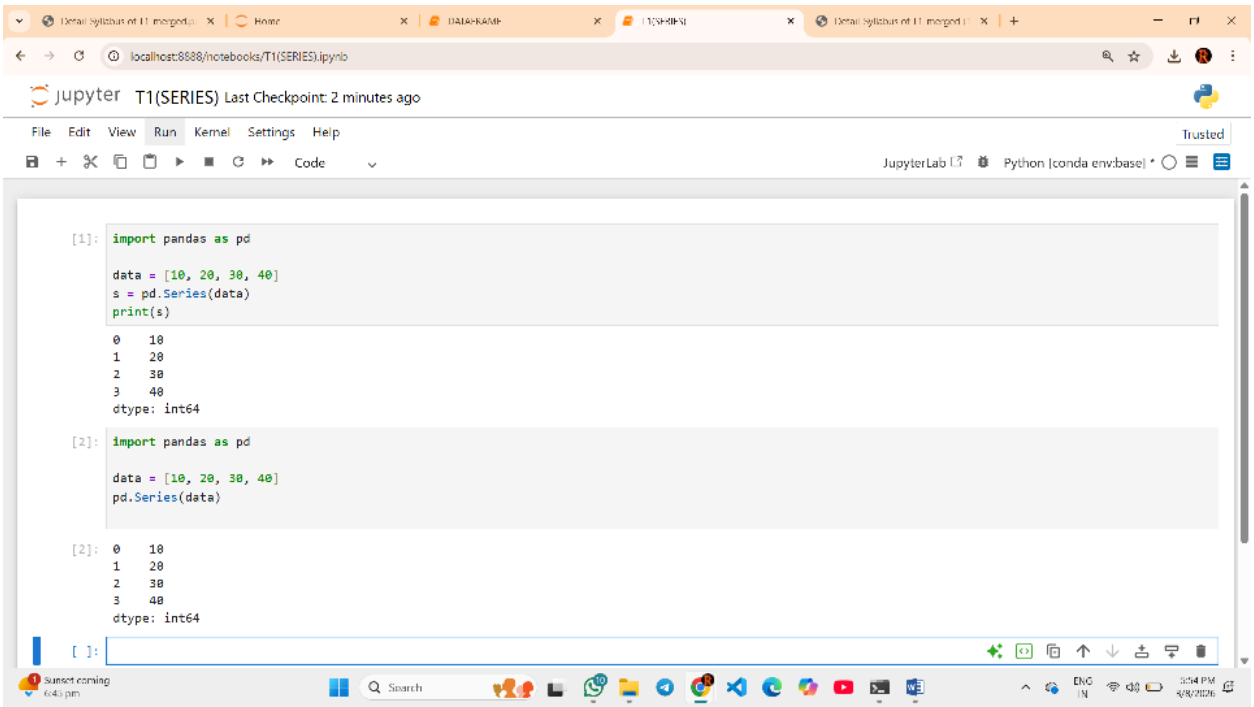
Pandas allows us to create a Series from different data structures because data can come from many sources. Converting them into a Series makes it easier to perform data analysis, filtering, and mathematical operations.

PRACTICAL PART :::

1. Series Using List

A list is one of the simplest ways to create a Pandas Series.

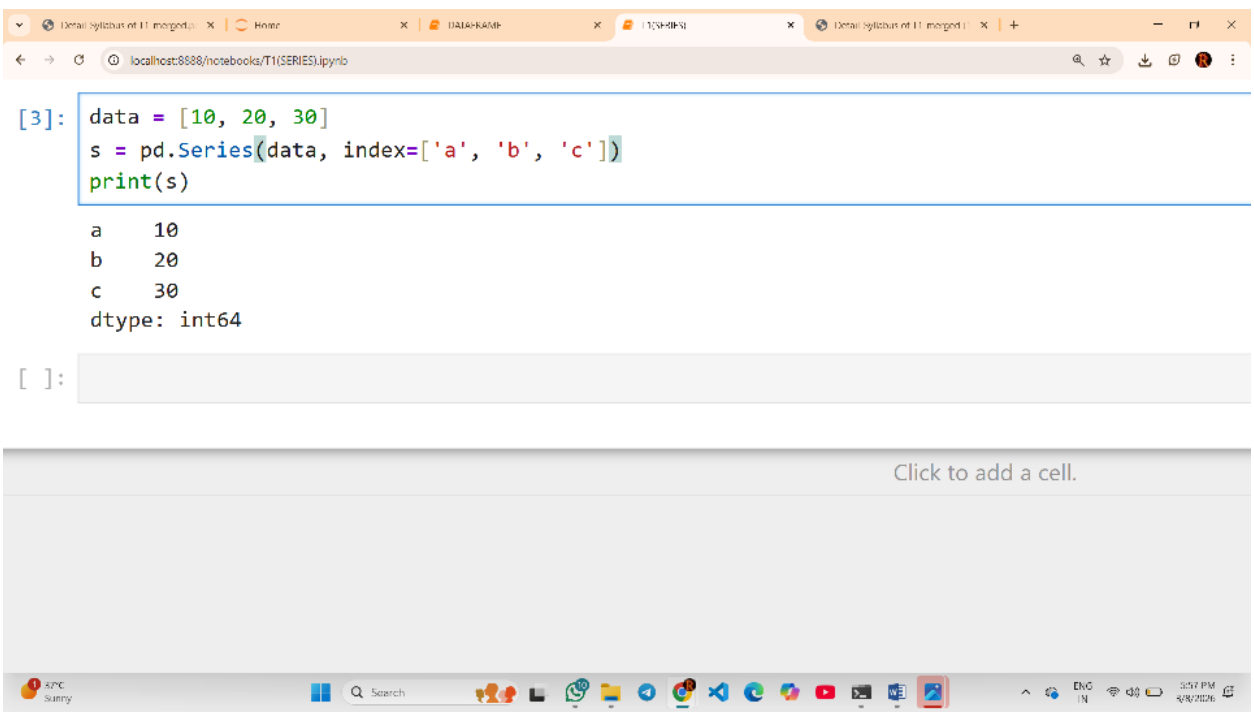
Each element of the list becomes a value in the Series, and by default Pandas assigns integer indexes starting from 0.



2. Series Using List with Custom Index

You can create a Series from a list and assign your own index labels instead of default numeric indexes.

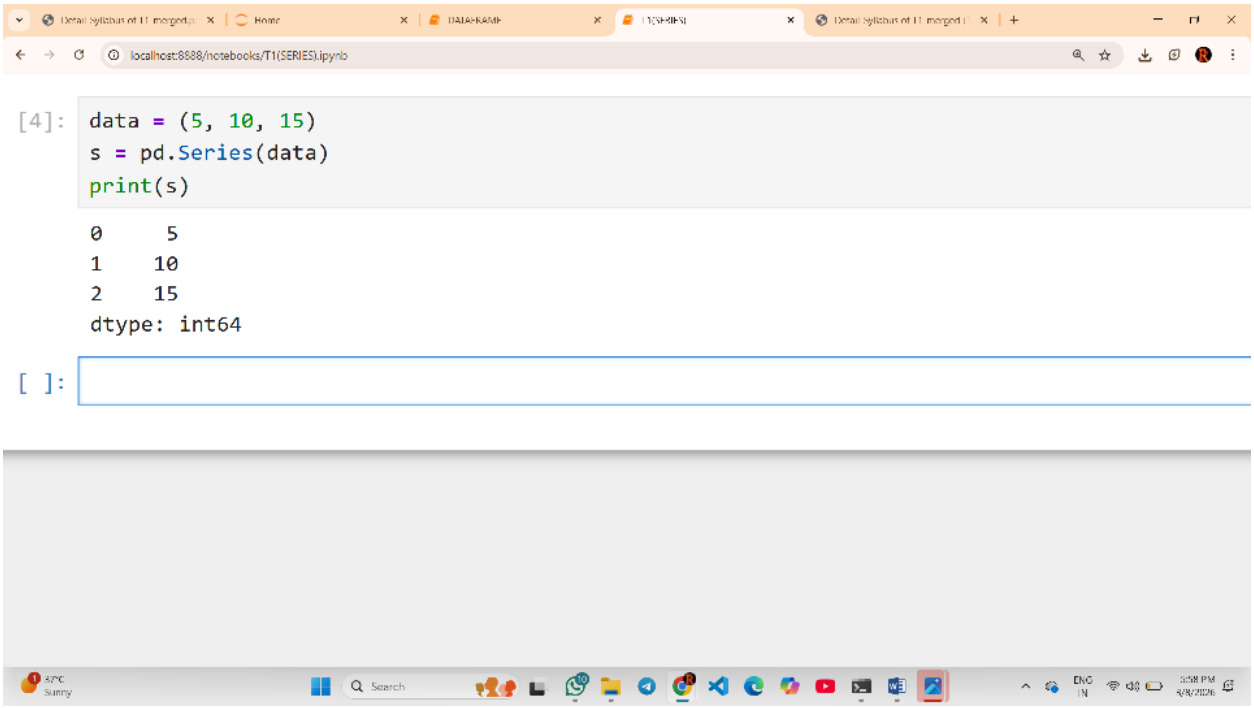
Example:



3. Series Using Tuple

A tuple can also be used to create a Series. Tuples are similar to lists but immutable (cannot be changed).

Example:



Useful when the dataset should remain constant and not be modified.

4. Series Using Dictionary

When creating a Series from a dictionary, the keys become the index and the values become the data.

Example:

```
[5]: data = {'a': 100, 'b': 200, 'c': 300}
     s = pd.Series(data)
     print(s)

     a    100
     b    200
     c    300
     dtype: int64

[ ]:
```

This method is useful when data already has labels associated with each value.

5. Series Using NumPy Array

A NumPy array can also be converted into a Pandas Series.

Example:

```
[6]: import numpy as np

     data = np.array([1, 2, 3, 4])
     s = pd.Series(data)
     print(s)

     0    1
     1    2
     2    3
     3    4
     dtype: int64

[ ]:
```

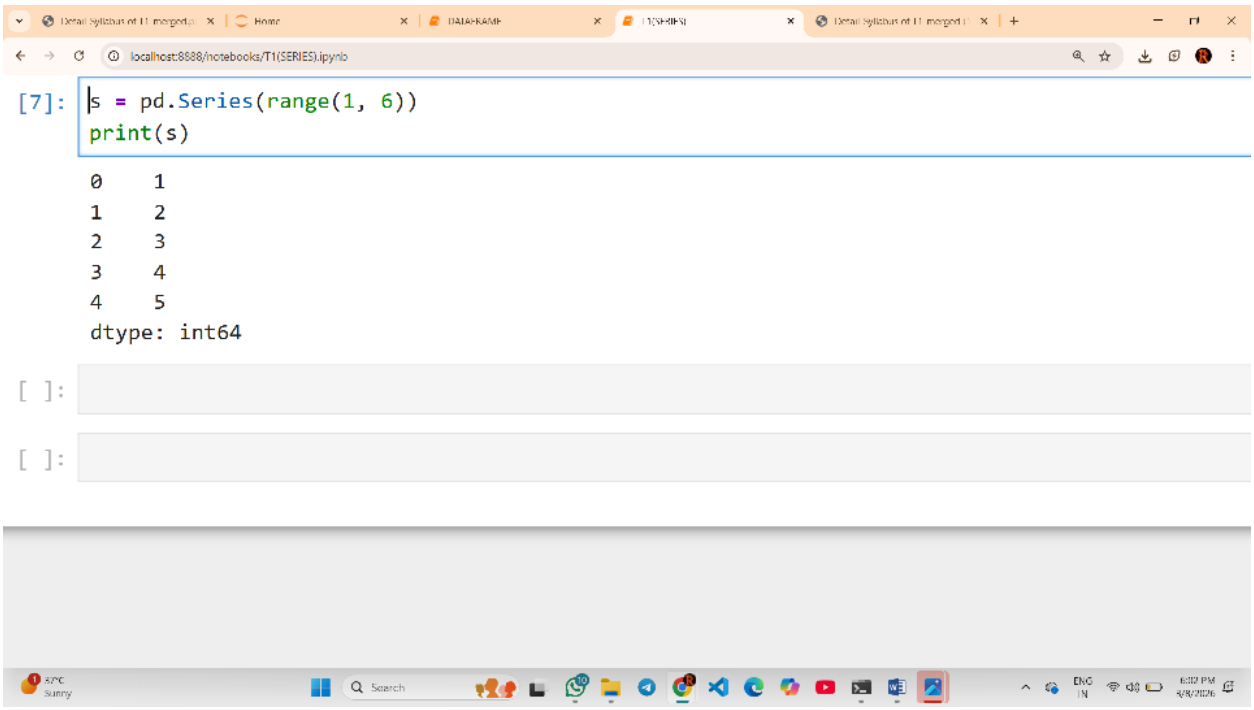
Intro:

This is commonly used in data science and numerical computing since NumPy handles large numerical data efficiently.

6. Series Using Range

The range() function generates a sequence of numbers which can be used to create a Series.

Example:



This method is helpful when you want to quickly generate sequential numeric data.

Practical Examples of Creating Pandas Series

1 Using List – Student Marks

Suppose you have marks of students stored in a list.

```
import pandas as pd


marks = [85, 90, 78, 88]

s = pd.Series(marks)

print(s)
```

Practical Example Explanation:

A teacher stores student marks in a list and converts it into a Series to analyze marks easily.

2 List with Custom Index – Student Names

Now suppose you want to show marks with student names.

```
marks = [85, 90, 78]

students = pd.Series(marks, index=["Rahul", "Priya", "Amit"])

print(students)
```

Practical Example Explanation:

This helps us clearly identify which student got which marks.

3 Using Tuple – Fixed Temperature Data

Suppose a weather station records fixed temperature values.

```
temp = (30, 32, 29, 31)

s = pd.Series(temp)

print(s)
```

Practical Example Explanation:

A tuple is used when the data should not change.

4 Using Dictionary – Product Prices

Suppose a shop stores product names and prices.

```
products = {

    "Laptop": 50000,

    "Mobile": 20000,

    "Tablet": 15000

}
```

```
s = pd.Series(products)

print(s)
```

Practical Example Explanation:

The product name becomes the index and price becomes the value.

5 Using NumPy Array – Scientific Data

In data science, data often comes in NumPy arrays.

```
import numpy as np


data = np.array([10, 20, 30, 40])

s = pd.Series(data)

print(s)
```

Practical Example Explanation:

Scientists convert NumPy numerical data into Pandas Series for analysis.

6 Using Range – Roll Numbers

Suppose you want to generate roll numbers automatically.

```
roll = pd.Series(range(1, 6))

print(roll)
```

Practical Example Explanation:

Used when we want sequential numbers quickly.

Exercise 1 – Series from List

Create a Pandas Series using the following list of student marks.

Data:

[65, 70, 75, 80, 85]

Task:

1. Import Pandas.
2. Create a Series from the list.
3. Display the Series.

Exercise 2 – Series with Custom Index

Create a Series for student marks with their names as index.

Data:

Marks:

[72, 88, 91]

Names:

["Rohit", "Anjali", "Karan"]

Task:

1. Create a Series.
2. Assign student names as the index.

Exercise 3 – Series from Tuple

Create a Pandas Series using the following tuple of temperatures.

Data:

(28, 30, 27, 29, 31)

Task:

1. Convert the tuple into a Pandas Series.
2. Display the Series.

Exercise 4 – Series from Dictionary

Create a Series using a dictionary of product prices.

Data:

{

"Pen": 10,

"Notebook": 50,

"Bag": 500

}

Task:

1. Convert the dictionary into a Series.
2. Display the output.

(Hint for students: dictionary keys will become indexes)

Exercise 5 – Series from Range

Create a Series using the range function.

Task:

1. Generate numbers from 1 to 10.
2. Convert them into a Pandas Series.
3. Print the Series.